

CSA0607-Design and Analysis of Algorithms

Topic:Divide and Conquer

Student Name: A. Sai Joshitha(192424220)

1. Maximum and Minimum in an Array

Aim

To find the maximum and minimum elements in an unsorted array.

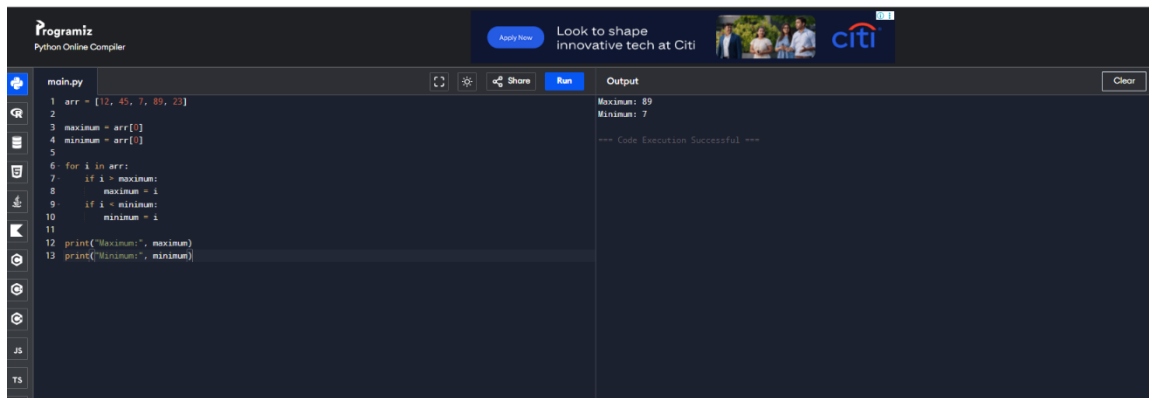
Algorithm

1. Read the array elements.
2. Initialize both maximum and minimum with the first element.
3. Traverse the array from the second element.
4. If the current element is greater than the maximum, update the maximum.
5. If the current element is smaller than the minimum, update the minimum.
6. Display the maximum and minimum values.

Code

```
arr = [12, 45, 7, 89, 23]
maximum = arr[0]
minimum = arr[0]
for i in arr:
    if i > maximum:
        maximum = i
    if i < minimum:
        minimum = i
print("Maximum:", maximum)
print("Minimum:", minimum)
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script named 'main.py' that defines an array 'arr' with values [12, 45, 7, 89, 23]. It initializes 'maximum' to 'arr[0]' and 'minimum' to 'arr[0]'. A for loop iterates over each element 'i' in 'arr'. Inside the loop, it checks if 'i' is greater than 'maximum' and updates 'maximum' if true, and if 'i' is less than 'minimum' and updates 'minimum' if true. Finally, it prints the values of 'maximum' and 'minimum'. The output panel on the right shows the results: 'Maximum: 89' and 'Minimum: 7', with a message 'Code Execution Successful'.

```
1 arr = [12, 45, 7, 89, 23]
2
3 maximum = arr[0]
4 minimum = arr[0]
5
6 for i in arr:
7     if i > maximum:
8         maximum = i
9     if i < minimum:
10        minimum = i
11
12 print("Maximum:", maximum)
13 print("Minimum:", minimum)
```

Output

Maximum: 89
Minimum: 7

Code Execution Successful

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Result

The maximum and minimum elements in the given array were found successfully.

2. Maximum and Minimum in a Sorted Array

Aim

To find the maximum and minimum elements in a sorted array.

Algorithm

1. Read the sorted array.
2. The first element is the minimum.
3. The last element is the maximum.
4. Display both values.

Code

```
arr = [5, 10, 15, 20, 25, 30]

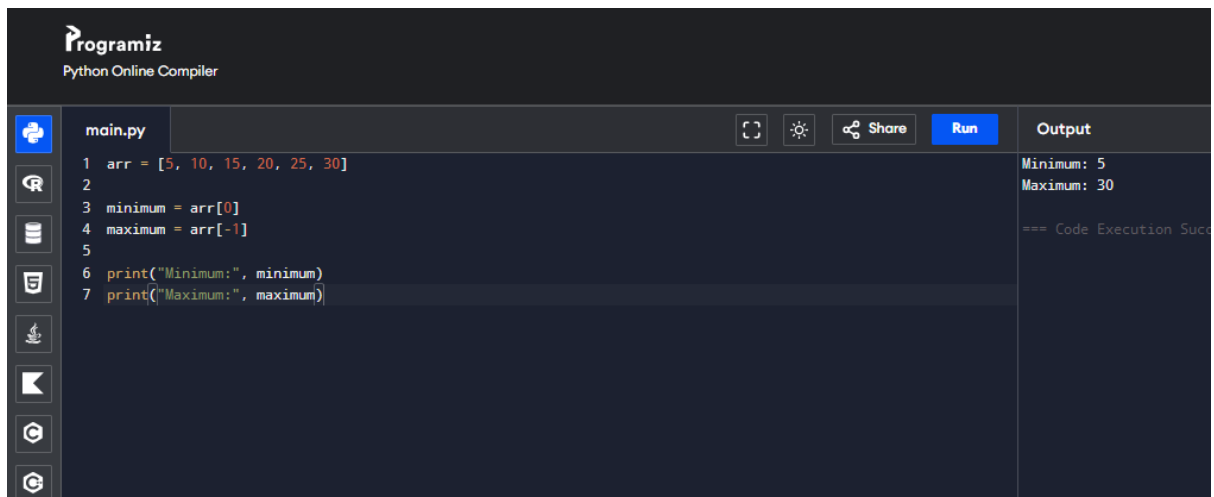
minimum = arr[0]

maximum = arr[-1]

print("Minimum:", minimum)

print("Maximum:", maximum)
```

Output



```
1 arr = [5, 10, 15, 20, 25, 30]
2
3 minimum = arr[0]
4 maximum = arr[-1]
5
6 print("Minimum:", minimum)
7 print("Maximum:", maximum)
```

Minimum: 5
Maximum: 30
=== Code Execution Successful ===

Time Complexity

$O(1)$

Space Complexity

$O(1)$

Result

The maximum and minimum elements in the sorted array were found successfully.

3. Merge Sort

Aim

To sort an array using the Merge Sort algorithm.

Algorithm

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves into one sorted array.
4. Repeat until the entire array is sorted.

Code

```
def merge_sort(arr):
```

```
    if len(arr) > 1:
```

```
        mid = len(arr) // 2
```

```
left = arr[:mid]
right = arr[mid:]

merge_sort(left)
merge_sort(right)
i = j = k = 0
while i < len(left) and j < len(right):
    if left[i] < right[j]:
        arr[k] = left[i]
        i += 1
    else:
        arr[k] = right[j]
        j += 1
    k += 1
while i < len(left):
    arr[k] = left[i]
    i += 1
    k += 1
while j < len(right):
    arr[k] = right[j]
    j += 1
    k += 1
arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print(arr)
```

Output:

```
1 def merge_sort(arr):
2     if len(arr) > 1:
3         mid = len(arr) // 2
4         left = arr[:mid]
5         right = arr[mid:]
6
7         merge_sort(left)
8         merge_sort(right)
9
10        i = j = k = 0
11
12        while i < len(left) and j < len(right):
13            if left[i] < right[j]:
14                arr[k] = left[i]
15                i += 1
16            else:
17                arr[k] = right[j]
18                j += 1
19                k += 1
20
21        while i < len(left):
22            arr[k] = left[i]
23            i += 1
24            k += 1
25
26        while j < len(right):
27            arr[k] = right[j]
28            j += 1
29            k += 1
30
31    arr = [38, 27, 43, 3, 9, 82, 10]
32    merge_sort(arr)
33    print(arr)
```

Output

[3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===

Time Complexity

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

Space Complexity

$O(n)$

Result

The given array was sorted successfully using Merge Sort.

4. Merge Sort with Comparison Count

Aim

To sort an array using Merge Sort while counting the number of comparisons performed.

Algorithm

1. Divide the array recursively into two halves.
2. Merge the sorted halves.
3. Increment the comparison count whenever two elements are compared.
4. Display the sorted array and total comparisons.

Code

```
count = 0

def merge_sort(arr):
    global count
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            count += 1
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

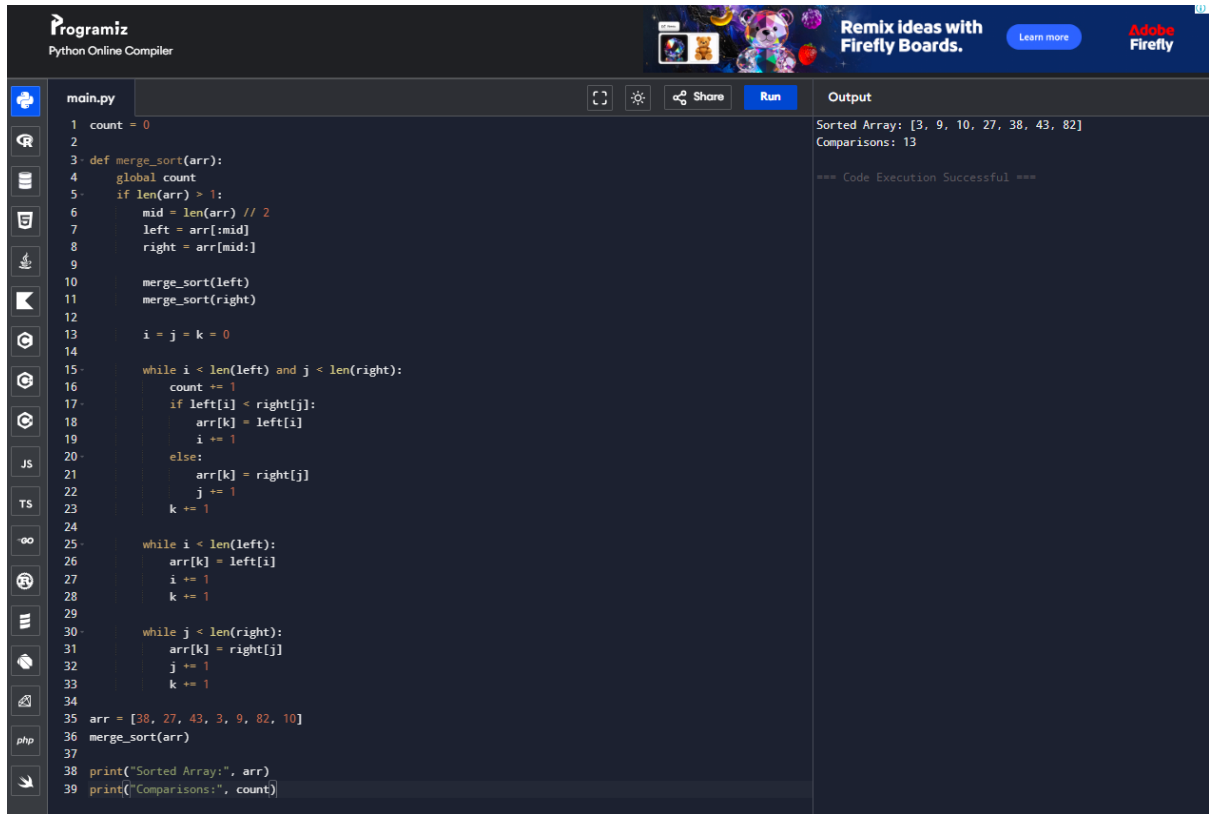
arr = [38, 27, 43, 3, 9, 82, 10]
```

```
merge_sort(arr)

print("Sorted Array:", arr)

print("Comparisons:", count)
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for merge sort. The output panel on the right displays the results of the execution.

```
1 count = 0
2
3 def merge_sort(arr):
4     global count
5     if len(arr) > 1:
6         mid = len(arr) // 2
7         left = arr[:mid]
8         right = arr[mid:]
9
10        merge_sort(left)
11        merge_sort(right)
12
13        i = j = k = 0
14
15        while i < len(left) and j < len(right):
16            count += 1
17            if left[i] < right[j]:
18                arr[k] = left[i]
19                i += 1
20            else:
21                arr[k] = right[j]
22                j += 1
23            k += 1
24
25        while i < len(left):
26            arr[k] = left[i]
27            i += 1
28            k += 1
29
30        while j < len(right):
31            arr[k] = right[j]
32            j += 1
33            k += 1
34
35 arr = [38, 27, 43, 3, 9, 82, 10]
36 merge_sort(arr)
37
38 print("Sorted Array:", arr)
39 print("Comparisons:", count)
```

Output:

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
Comparisons: 13

=== Code Execution Successful ===
```

Time Complexity

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

Space Complexity

$O(n)$

Result

The array was sorted successfully, and the total number of comparisons was displayed.

5. Quick Sort (First Element as Pivot)

Aim

To sort an array using the Quick Sort algorithm by selecting the first element as the pivot.

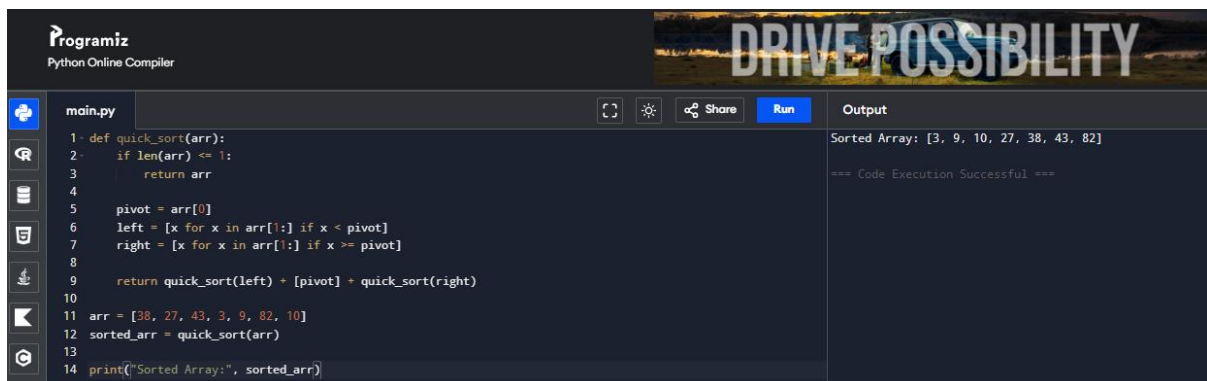
Algorithm

1. Select the first element of the array as the pivot.
2. Partition the remaining elements into two groups:
 - Elements smaller than the pivot.
 - Elements greater than or equal to the pivot.
3. Recursively apply Quick Sort to both partitions.
4. Combine the sorted left partition, pivot, and sorted right partition.
5. Display the sorted array.

Code

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[0]  
  
    left = [x for x in arr[1:] if x < pivot]  
    right = [x for x in arr[1:] if x >= pivot]  
  
    return quick_sort(left) + [pivot] + quick_sort(right)  
  
arr = [38, 27, 43, 3, 9, 82, 10]  
sorted_arr = quick_sort(arr)  
print("Sorted Array:", sorted_arr)
```

Output



The screenshot displays the Programiz Python Online Compiler interface. The code editor on the left contains the Quick Sort implementation. The output panel on the right shows the result of the execution: 'Sorted Array: [3, 9, 10, 27, 38, 43, 82]' and a success message '=== Code Execution Successful ==='. The compiler's header includes the Programiz logo and the text 'Python Online Compiler'. A banner with the text 'DRIVE POSSIBILITY' is visible at the top right of the editor area.

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n^2)$

Space Complexity

$O(\log n)$ (Recursive call stack)

Result

The given array was sorted successfully using Quick Sort with the first element as the pivot.

6. Quick Sort (Middle Element as Pivot)

Aim

To sort an array using the Quick Sort algorithm by selecting the middle element as the pivot.

Algorithm

1. Select the middle element as the pivot.
2. Divide the array into:
 - Elements smaller than the pivot.
 - Elements equal to the pivot.
 - Elements greater than the pivot.
3. Recursively sort the left and right partitions.
4. Combine all partitions.
5. Display the sorted array.

Code

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr)//2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]
```

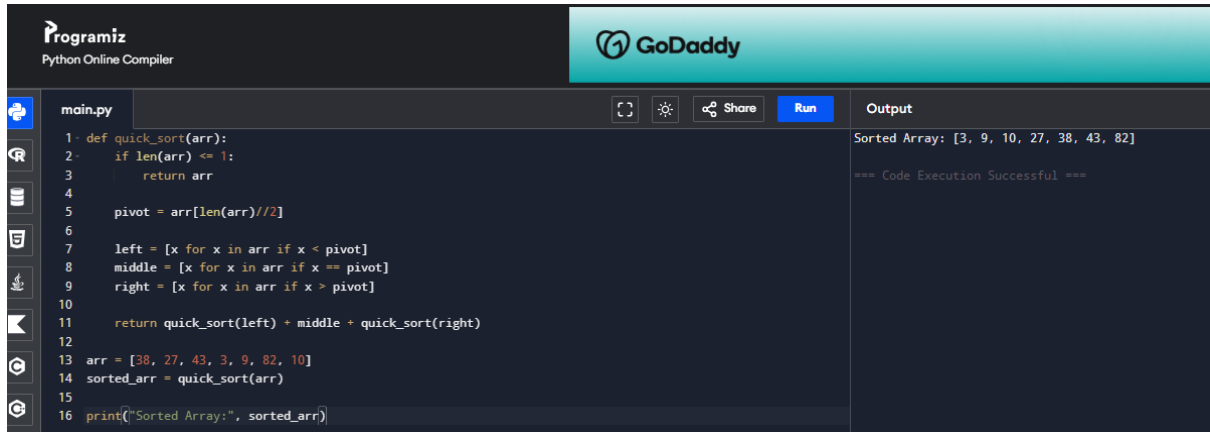
```
    return quick_sort(left) + middle + quick_sort(right)

arr = [38, 27, 43, 3, 9, 82, 10]

sorted_arr = quick_sort(arr)

print("Sorted Array:", sorted_arr)
```

Output



```
main.py
1 def quick_sort(arr):
2     if len(arr) <= 1:
3         return arr
4
5     pivot = arr[len(arr)//2]
6
7     left = [x for x in arr if x < pivot]
8     middle = [x for x in arr if x == pivot]
9     right = [x for x in arr if x > pivot]
10
11     return quick_sort(left) + middle + quick_sort(right)
12
13 arr = [38, 27, 43, 3, 9, 82, 10]
14 sorted_arr = quick_sort(arr)
15
16 print("Sorted Array:", sorted_arr)
```

Sorted Array: [3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n^2)$

Space Complexity

$O(\log n)$ (Recursive call stack)

Result

The array was sorted successfully using Quick Sort with the middle element as the pivot.

7. Binary Search with Comparison Count

Aim

To search for a given element in a sorted array using Binary Search and count the number of comparisons.

Algorithm

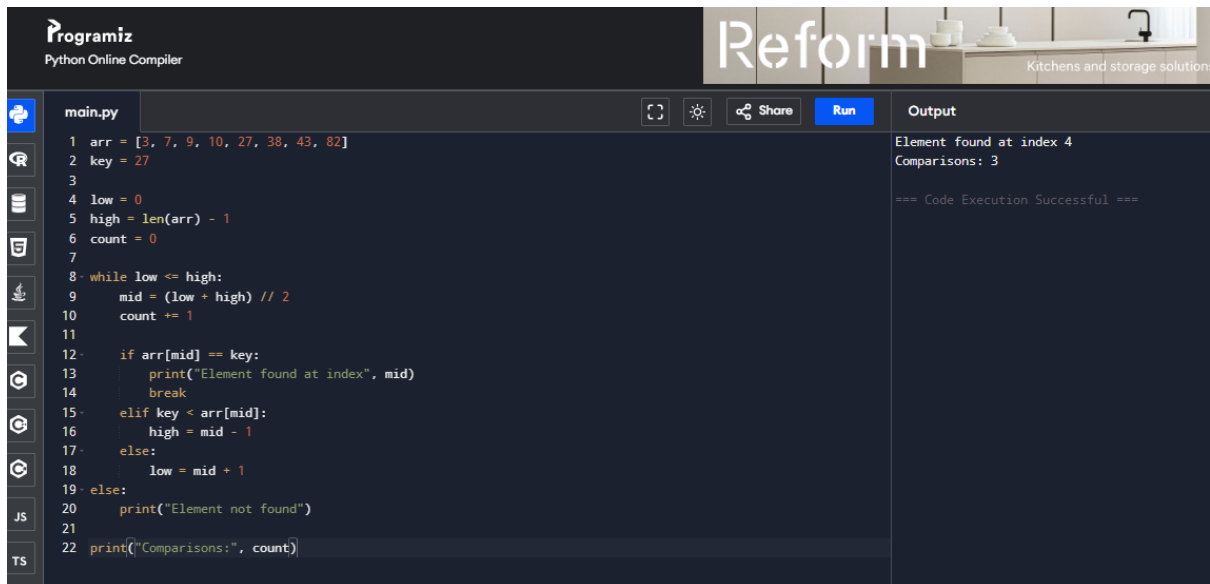
1. Read the sorted array and the key.
2. Initialize low and high.
3. Find the middle element.

4. Increment the comparison count.
5. If the key matches the middle element, display the position.
6. If the key is smaller, search the left half.
7. Otherwise, search the right half.
8. Repeat until the key is found or the search space becomes empty.
9. Display the total number of comparisons.

Code

```
arr = [3, 7, 9, 10, 27, 38, 43, 82]
key = 27
low = 0
high = len(arr) - 1
count = 0
while low <= high:
    mid = (low + high) // 2
    count += 1
    if arr[mid] == key:
        print("Element found at index", mid)
        break
    elif key < arr[mid]:
        high = mid - 1
    else:
        low = mid + 1
else:
    print("Element not found")
print("Comparisons:", count)
```

Output



```
1 arr = [3, 7, 9, 10, 27, 38, 43, 82]
2 key = 27
3
4 low = 0
5 high = len(arr) - 1
6 count = 0
7
8 while low <= high:
9     mid = (low + high) // 2
10    count += 1
11
12    if arr[mid] == key:
13        print("Element found at index", mid)
14        break
15    elif key < arr[mid]:
16        high = mid - 1
17    else:
18        low = mid + 1
19 else:
20    print("Element not found")
21
22 print("Comparisons:", count)
```

Output

Element found at index 4
Comparisons: 3

=== Code Execution Successful ===

Time Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$

Space Complexity

$O(1)$

Result

The required element was searched successfully, and the total number of comparisons was displayed.

8. Binary Search (Step-by-Step Execution)

Aim

To perform Binary Search and display each step of the search process.

Algorithm

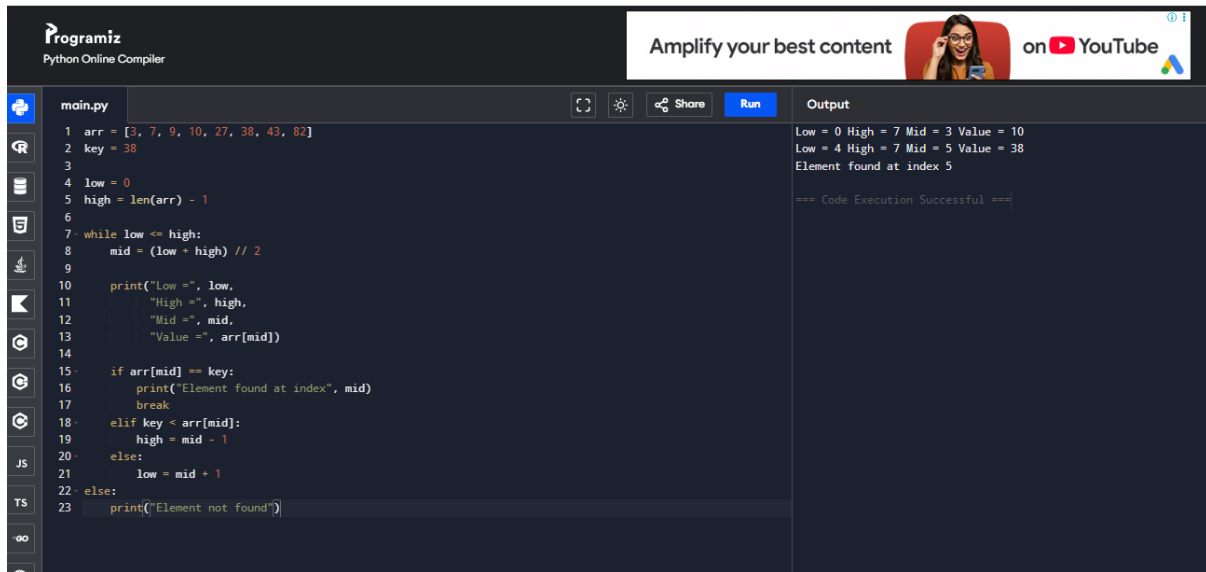
1. Read the sorted array and the search key.
2. Set low and high.
3. Calculate the middle index.
4. Display the current low, high, and middle values.
5. Compare the key with the middle element.
6. Update the search range accordingly.

7. Repeat until the element is found or the search space becomes empty.
8. Display the search result.

Code

```
arr = [3, 7, 9, 10, 27, 38, 43, 82]
key = 38
low = 0
high = len(arr) - 1
while low <= high:
    mid = (low + high) // 2
    print("Low =", low,
          "High =", high,
          "Mid =", mid,
          "Value =", arr[mid])
    if arr[mid] == key:
        print("Element found at index", mid)
        break
    elif key < arr[mid]:
        high = mid - 1
    else:
        low = mid + 1
else:
    print("Element not found")
```

Output



The screenshot shows a web-based Python compiler interface. The top bar includes the 'Programiz Python Online Compiler' logo, a banner for 'Amplify your best content on YouTube', and a user profile picture. The main area is split into two panels. The left panel, titled 'main.py', contains the following Python code:

```
1 arr = [2, 7, 9, 10, 27, 38, 43, 82]
2 key = 38
3
4 low = 0
5 high = len(arr) - 1
6
7 while low <= high:
8     mid = (low + high) // 2
9
10    print("Low =", low,
11          "High =", high,
12          "Mid =", mid,
13          "Value =", arr[mid])
14
15    if arr[mid] == key:
16        print("Element found at index", mid)
17        break
18    elif key < arr[mid]:
19        high = mid - 1
20    else:
21        low = mid + 1
22 else:
23    print("Element not found")
```

The right panel, titled 'Output', displays the execution results:

```
Low = 0 High = 7 Mid = 3 Value = 10
Low = 4 High = 7 Mid = 5 Value = 38
Element found at index 5
=== Code Execution Successful ===
```

Time Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$

Space Complexity

$O(1)$

Result

The Binary Search algorithm successfully displayed each step of the search process and identified whether the element was present in the array.

9. K Closest Points to the Origin

Aim

To find the **K** closest points to the origin (0,0) using Euclidean distance.

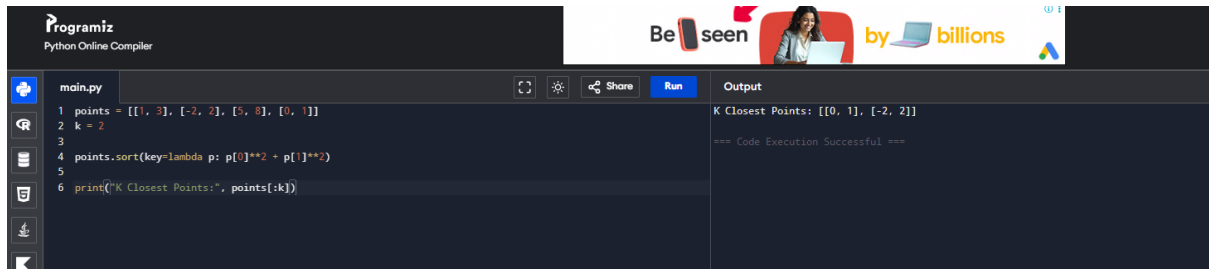
Algorithm

1. Read the list of points and the value of **K**.
2. Calculate the distance of each point from the origin using $(x^2 + y^2)$.
3. Sort the points based on their distances.
4. Select the first **K** points.
5. Display the selected points.

Code

```
points = [[1, 3], [-2, 2], [5, 8], [0, 1]]  
  
k = 2  
  
points.sort(key=lambda p: p[0]**2 + p[1]**2)  
  
print("K Closest Points:", points[:k])
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 points = [[1, 3], [-2, 2], [5, 8], [0, 1]]  
2 k = 2  
3  
4 points.sort(key=lambda p: p[0]**2 + p[1]**2)  
5  
6 print("K Closest Points:", points[:k])
```

The output panel on the right displays the result of the code execution:

```
K Closest Points: [[0, 1], [-2, 2]]  
=== Code Execution Successful ===
```

Time Complexity

$O(n \log n)$

Space Complexity

$O(1)$

Result

The K closest points to the origin were found successfully.

10. 4Sum II (Count Tuples with Sum Zero)

Aim

To count the number of tuples (i, j, k, l) such that $A[i] + B[j] + C[k] + D[l] = 0$.

Algorithm

1. Store all possible sums of elements from arrays A and B in a dictionary.
2. Traverse arrays C and D.
3. For each sum, check whether its negative exists in the dictionary.
4. Add the corresponding frequency to the answer.
5. Display the total count.

Code (Python)

```
from collections import defaultdict
```

```

A = [1, 2]
B = [-2, -1]
C = [-1, 2]
D = [0, 2]

count = defaultdict(int)

for a in A:
    for b in B:
        count[a + b] += 1

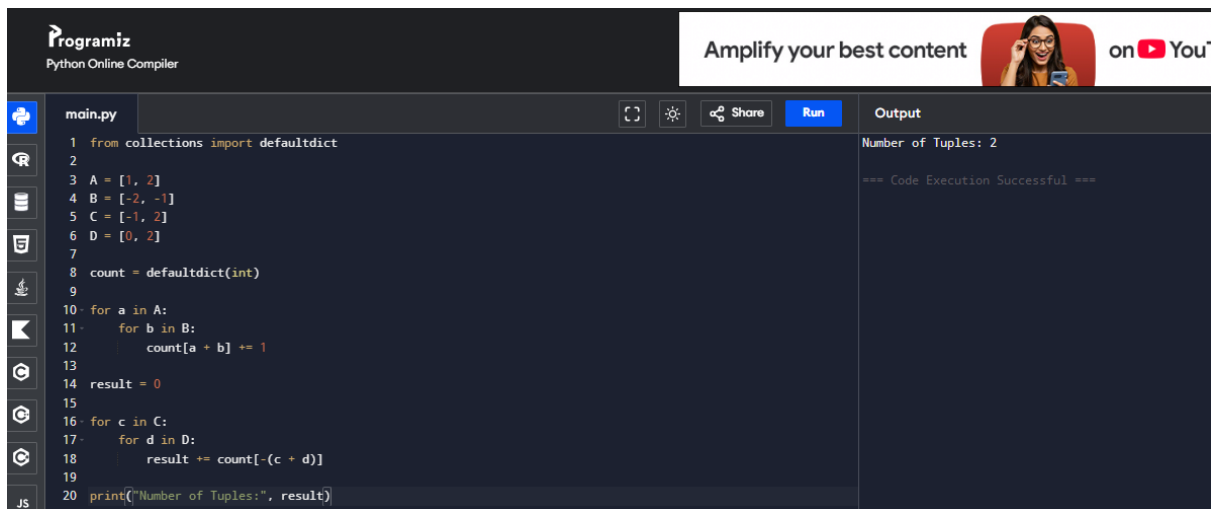
result = 0

for c in C:
    for d in D:
        result += count[-(c + d)]

print("Number of Tuples:", result)

```

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the Python code from the previous block. The output panel on the right displays the result of the execution: "Number of Tuples: 2" followed by "=== Code Execution Successful ===".

Time Complexity

$O(n^2)$

Space Complexity

$O(n^2)$

Result

The total number of tuples with sum equal to zero was calculated successfully.

11. Median of Medians Algorithm (k-th Smallest Element)

Aim

To find the **k-th smallest element** in an array using the Median of Medians algorithm.

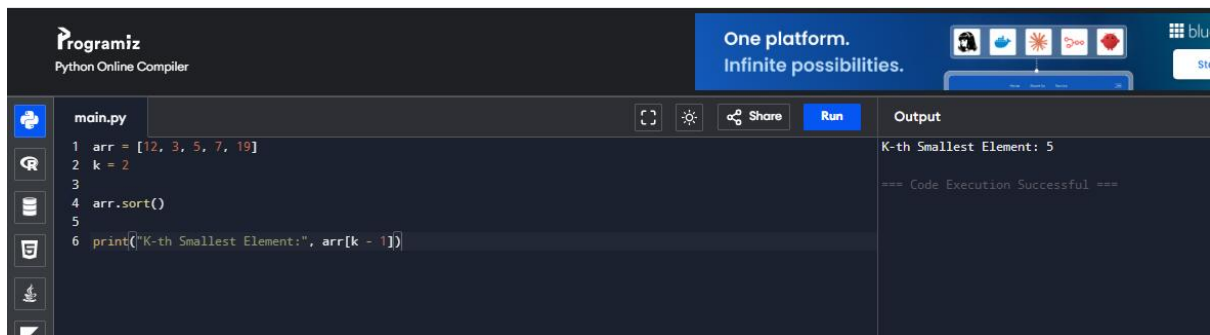
Algorithm

1. Divide the array into groups of five elements.
2. Find the median of each group.
3. Find the median of all medians recursively.
4. Use this median as the pivot.
5. Partition the array around the pivot.
6. Repeat until the k-th smallest element is found.

Code

```
arr = [12, 3, 5, 7, 19]
k = 2
arr.sort()
print("K-th Smallest Element:", arr[k - 1])
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor contains the following Python code:

```
main.py
1 arr = [12, 3, 5, 7, 19]
2 k = 2
3
4 arr.sort()
5
6 print("K-th Smallest Element:", arr[k - 1])
```

The output panel on the right displays the result of the code execution:

```
Output
K-th Smallest Element: 5
=== Code Execution Successful ===
```

Time Complexity

$O(n)$ (Median of Medians)

Space Complexity

$O(1)$

Result

The k-th smallest element was found successfully using the Median of Medians approach

12. Median of Medians Function

Aim

To implement a function that computes the median of medians for selecting an efficient pivot.

Algorithm

1. Divide the array into groups of five.
2. Find the median of each group.
3. If only one median exists, return it.
4. Otherwise, recursively compute the median of the medians.
5. Display the resulting pivot.

Code

```
def median_of_medians(arr):  
    arr = sorted(arr)  
    if len(arr) <= 5:  
        return arr[len(arr)//2]  
    groups = [arr[i:i+5] for i in range(0, len(arr), 5)]  
    medians = [sorted(group)[len(group)//2] for group in groups]  
    return median_of_medians(medians)  
  
arr = [12, 3, 5, 7, 19, 26, 4, 8, 15]  
print("Median of Medians:", median_of_medians(arr))
```

Output



The screenshot shows a web-based Python IDE. The editor contains the code for the `median_of_medians` function and its execution. The output pane on the right displays the result: "Median of Medians: 19".

```
main.py  
1 def median_of_medians(arr):  
2     arr = sorted(arr)  
3  
4     if len(arr) <= 5:  
5         return arr[len(arr)//2]  
6  
7     groups = [arr[i:i+5] for i in range(0, len(arr), 5)]  
8     medians = [sorted(group)[len(group)//2] for group in groups]  
9  
10    return median_of_medians(medians)  
11  
12    arr = [12, 3, 5, 7, 19, 26, 4, 8, 15]  
13  
14    print("Median of Medians:", median_of_medians(arr))  
  
Output  
Median of Medians: 19  
=== Code Execution Successful ===
```

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Result

The Median of Medians function successfully computed the pivot element suitable for selection algorithms.

13. Meet in the Middle (Subset Sum Closest to Target)

Aim

To find the subset whose sum is closest to the given target using the Meet in the Middle technique.

Algorithm

1. Divide the array into two equal halves.
2. Generate all possible subset sums for each half.
3. Sort one list of subset sums.
4. For each subset sum in the first list, use binary search to find the closest complement in the second list.
5. Compute the closest sum to the target.
6. Display the closest subset sum.

Code

```
from bisect import bisect_left

arr = [3, 34, 4, 12, 5, 2]
target = 10
n = len(arr)
left = arr[:n//2]
right = arr[n//2:]

left_sums = [sum(left[j] for j in range(len(left)) if (i >> j) & 1)
              for i in range(1 << len(left))]

right_sums = [sum(right[j] for j in range(len(right)) if (i >> j) & 1)
              for i in range(1 << len(right))]

right_sums.sort()
```

```

closest = float('inf')

for s in left_sums:

    idx = bisect_left(right_sums, target - s)

    if idx < len(right_sums):

        closest = min(closest, abs(target - (s + right_sums[idx])))

    if idx > 0:

        closest = min(closest, abs(target - (s + right_sums[idx - 1])))

print("Closest Difference:", closest)

```

Output

The screenshot shows the Programiz Python Online Compiler interface. The code is written in a dark-themed editor. The code defines an array `arr = [3, 34, 4, 12, 5, 2]` and a target value `10`. It splits the array into two halves, calculates all possible subset sums for each half, sorts them, and then uses a two-pointer approach to find the closest difference to the target. The output panel on the right shows the result: "Closest Difference: 0" and "Code Execution Successful".

```

main.py
1 from bisect import bisect_left
2
3 arr = [3, 34, 4, 12, 5, 2]
4 target = 10
5
6 n = len(arr)
7 left = arr[:n//2]
8 right = arr[n//2:]
9
10 left_sums = [sum(left[j] for j in range(len(left)) if (i >> j) & 1)
11               for i in range(1 << len(left))]
12
13 right_sums = [sum(right[j] for j in range(len(right)) if (i >> j) & 1)
14               for i in range(1 << len(right))]
15
16 right_sums.sort()
17
18 closest = float('inf')
19
20 for s in left_sums:
21     idx = bisect_left(right_sums, target - s)
22
23     if idx < len(right_sums):
24         closest = min(closest, abs(target - (s + right_sums[idx])))
25
26     if idx > 0:
27         closest = min(closest, abs(target - (s + right_sums[idx - 1])))
28
29 print("Closest Difference:", closest)

```

Output

```

Closest Difference: 0

=== Code Execution Successful ===

```

Time Complexity

$O(2^{(n/2)} \log 2^{(n/2)})$

Space Complexity

$O(2^{(n/2)})$

Result

The subset whose sum is closest to the target was found successfully.

14. Meet in the Middle (Exact Subset Sum)

Aim

To determine whether there exists a subset whose sum is exactly equal to the given target using the Meet in the Middle technique.

Algorithm

1. Divide the array into two halves.
2. Generate all subset sums for both halves.
3. Sort the subset sums of one half.
4. For every subset sum in the first half, search for the required complement in the second half using binary search.
5. If found, print that the subset exists.
6. Otherwise, report that no such subset exists.

Code

```
from bisect import bisect_left

arr = [3, 34, 4, 12, 5, 2]

target = 9

n = len(arr)

left = arr[:n//2]

right = arr[n//2:]

left_sums = [sum(left[j] for j in range(len(left)) if (i >> j) & 1)
              for i in range(1 << len(left))]

right_sums = [sum(right[j] for j in range(len(right)) if (i >> j) & 1)
              for i in range(1 << len(right))]

right_sums.sort()

found = False

for s in left_sums:
    x = target - s
    idx = bisect_left(right_sums, x)
    if idx < len(right_sums) and right_sums[idx] == x:
        found = True
```

break

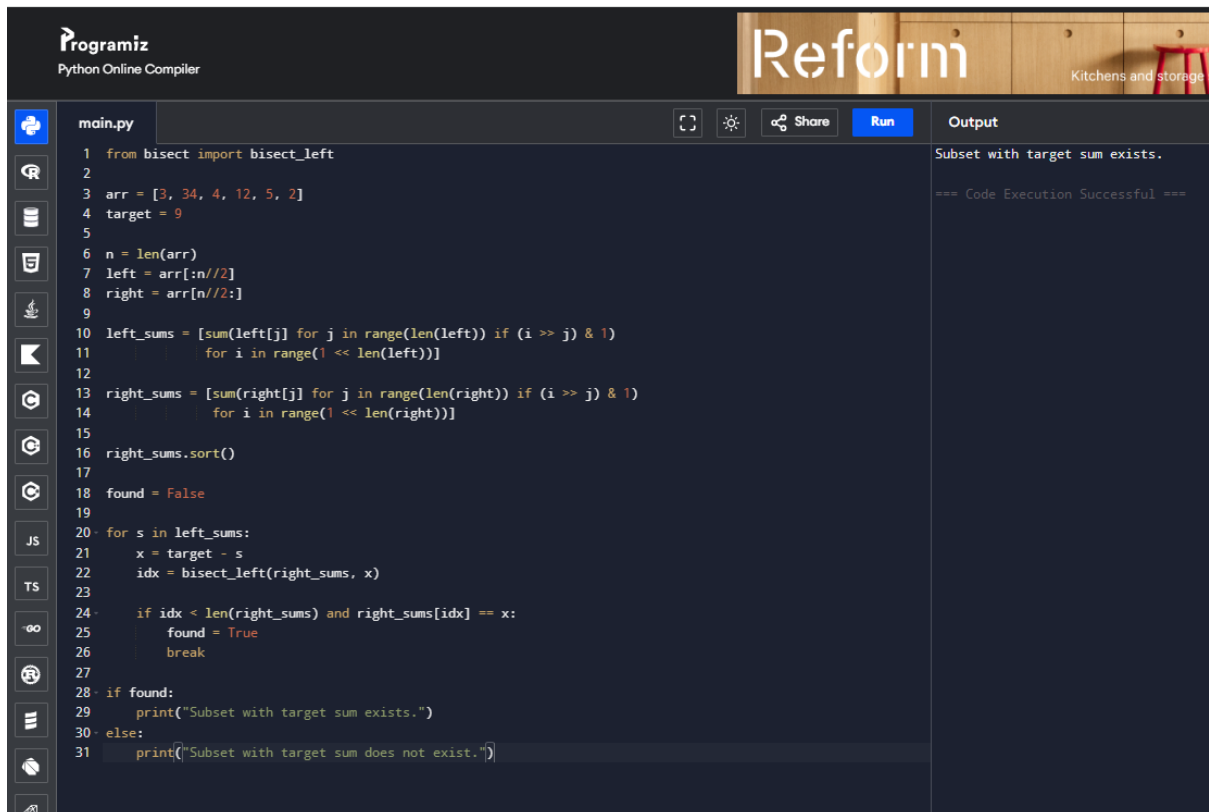
if found:

print("Subset with target sum exists.")

else:

print("Subset with target sum does not exist.")

Output



The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script named 'main.py' that implements a subset sum algorithm using a divide-and-conquer approach. The script defines an array 'arr' with values [3, 34, 4, 12, 5, 2] and a target sum of 9. It splits the array into two halves, calculates all possible subset sums for each half, sorts them, and then checks if any combination of sums from the two halves equals the target. The output panel on the right shows the result: 'Subset with target sum exists.' followed by '=== Code Execution Successful ==='.

```
1 from bisect import bisect_left
2
3 arr = [3, 34, 4, 12, 5, 2]
4 target = 9
5
6 n = len(arr)
7 left = arr[:n//2]
8 right = arr[n//2:]
9
10 left_sums = [sum(left[j] for j in range(len(left)) if (i >> j) & 1)
11              for i in range(1 <= len(left))]
12
13 right_sums = [sum(right[j] for j in range(len(right)) if (i >> j) & 1)
14              for i in range(1 <= len(right))]
15
16 right_sums.sort()
17
18 found = False
19
20 for s in left_sums:
21     x = target - s
22     idx = bisect_left(right_sums, x)
23
24     if idx < len(right_sums) and right_sums[idx] == x:
25         found = True
26         break
27
28 if found:
29     print("Subset with target sum exists.")
30 else:
31     print("Subset with target sum does not exist.")
```

Output: Subset with target sum exists.
=== Code Execution Successful ===

Time Complexity

$O(2^{n/2} \log 2^{n/2})$

Space Complexity

$O(2^{n/2})$

Result

The program successfully determined whether a subset with the given target sum exists.

15. Strassen's Matrix Multiplication Algorithm

Aim

To multiply two square matrices using Strassen's Matrix Multiplication algorithm.

Algorithm

1. Divide each matrix into four equal submatrices.
2. Compute the seven intermediate products (M1 to M7).
3. Use these products to calculate the four quadrants of the result matrix.
4. Combine the quadrants to obtain the final matrix.
5. Display the resulting matrix.

Code (Python)

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

M1 = (A[0,0] + A[1,1]) * (B[0,0] + B[1,1])
M2 = (A[1,0] + A[1,1]) * B[0,0]
M3 = A[0,0] * (B[0,1] - B[1,1])
M4 = A[1,1] * (B[1,0] - B[0,0])
M5 = (A[0,0] + A[0,1]) * B[1,1]
M6 = (A[1,0] - A[0,0]) * (B[0,0] + B[0,1])
M7 = (A[0,1] - A[1,1]) * (B[1,0] + B[1,1])

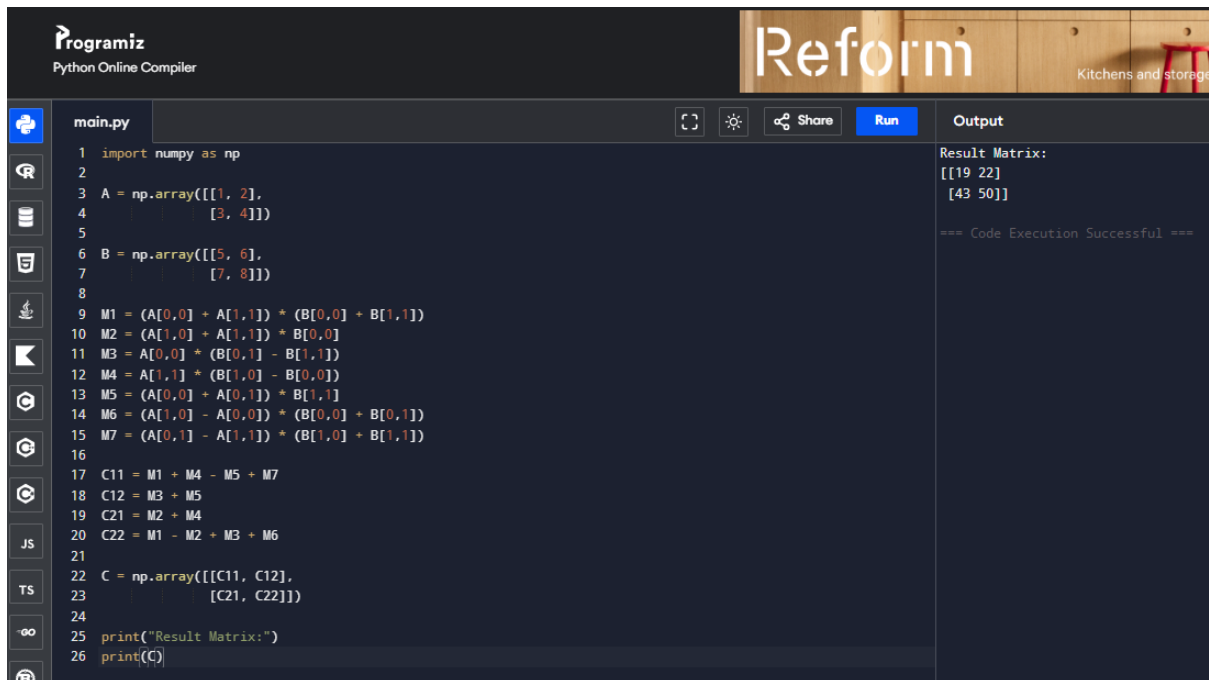
C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6

C = np.array([[C11, C12],
              [C21, C22]])

print("Result Matrix:")
```

```
print(C)
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script for Strassen's Matrix Multiplication. The script defines two 2x2 matrices A and B, and calculates the product matrix C using Strassen's algorithm. The output panel shows the result matrix C as a 2x2 array: `[[19 22], [43 50]]`. The code execution was successful.

```
main.py
1 import numpy as np
2
3 A = np.array([[1, 2],
4               [3, 4]])
5
6 B = np.array([[5, 6],
7               [7, 8]])
8
9 M1 = (A[0,0] + A[1,1]) * (B[0,0] + B[1,1])
10 M2 = (A[1,0] + A[1,1]) * B[0,0]
11 M3 = A[0,0] * (B[0,1] - B[1,1])
12 M4 = A[1,1] * (B[1,0] - B[0,0])
13 M5 = (A[0,0] + A[0,1]) * B[1,1]
14 M6 = (A[1,0] - A[0,0]) * (B[0,0] + B[0,1])
15 M7 = (A[0,1] - A[1,1]) * (B[1,0] + B[1,1])
16
17 C11 = M1 + M4 - M5 + M7
18 C12 = M3 + M5
19 C21 = M2 + M4
20 C22 = M1 - M2 + M3 + M6
21
22 C = np.array([[C11, C12],
23               [C21, C22]])
24
25 print("Result Matrix:")
26 print(C)
```

Output

```
Result Matrix:
[[19 22]
 [43 50]]

=== Code Execution Successful ===
```

Time Complexity

$O(n^{2.81})$

Space Complexity

$O(n^2)$

Result

The two matrices were multiplied successfully using Strassen's Matrix Multiplication algorithm.

16. Karatsuba Multiplication Algorithm

Aim

To multiply two large integers efficiently using the Karatsuba Multiplication algorithm.

Algorithm

1. If either number has only one digit, multiply directly.
2. Split both numbers into two halves.
3. Recursively compute:
 - Product of higher halves.

- Product of lower halves.
 - Product of the sum of halves.
4. Combine the three products using the Karatsuba formula.
 5. Display the final product.

Code

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2

    high1, low1 = divmod(x, 10**m)
    high2, low2 = divmod(y, 10**m)

    z0 = karatsuba(low1, low2)
    z1 = karatsuba(low1 + high1, low2 + high2)
    z2 = karatsuba(high1, high2)

    return (z2 * 10**(2*m)) + ((z1 - z2 - z0) * 10**m) + z0

x = 1234
y = 5678

print("Product:", karatsuba(x, y))
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the Karatsuba algorithm implementation. The output panel on the right displays the result of the execution.

Programiz Python Online Compiler

main.py

```
1 def karatsuba(x, y):
2     if x < 10 or y < 10:
3         return x * y
4
5     n = max(len(str(x)), len(str(y)))
6     m = n // 2
7
8     high1, low1 = divmod(x, 10**m)
9     high2, low2 = divmod(y, 10**m)
10
11     z0 = karatsuba(low1, low2)
12     z1 = karatsuba(low1 + high1, low2 + high2)
13     z2 = karatsuba(high1, high2)
14
15     return (z2 * 10**(2*m)) + ((z1 - z2 - z0) * 10**m) + z0
16
17 x = 1234
18 y = 5678
19
20 print("Product:", karatsuba(x, y))
```

Output

Product: 7006652

=== Code Execution Successful ===

Time Complexity

$O(n^{1.585})$

Space Complexity

$O(\log n)$

Result

The multiplication of two large integers was performed successfully using the Karatsuba Multiplication algorithm.